

ローカルミニマムの罠

初期値しだいで答えが変わる

rkskmt

1

本編の振り返りと今回の問題

2

本編でやったこと

本編では $f(x) = x^2 - 4x + 3$ という **2次関数** で、勾配降下法（と参考資料の **Newton法**）を学びました。

この関数は **谷が1つだけ** のシンプルな形でした。どこから始めても、必ず同じ最小値 $x = 2$ にたどり着きました。

① 本編の結論

- 勾配降下法：何回も反復して少しずつ最小値に近づく
- Newton法：2次微分も使って、一気に最小値にジャンプ
- どちらも **同じ答え** にたどり着いた

3

今回の疑問

でも、こんな疑問が浮かびます：

「**谷が2つ以上ある関数だったら、どうなるの？**」

実際の機械学習の問題では、谷が1つだけということはほとんどありません。パラメータが多くなると、**たくさんの谷** が存在します。

今回は「谷が2つある関数」で、Newton法を使ったときに **何が起こるか** を体験しましょう。

4

谷が2つある関数を見てみよう

5

関数の定義

今回使う関数はこれです：

$$f(x) = 0.1x^4 - x^3 + 2x^2 + 1$$

本編と同じように、微分を手で計算しておきます。

1次微分（傾き）：

$$f'(x) = 0.4x^3 - 3x^2 + 4x$$

2次微分（曲がり具合）：

$$f''(x) = 1.2x^2 - 6x + 4$$

6

Pythonでの定義

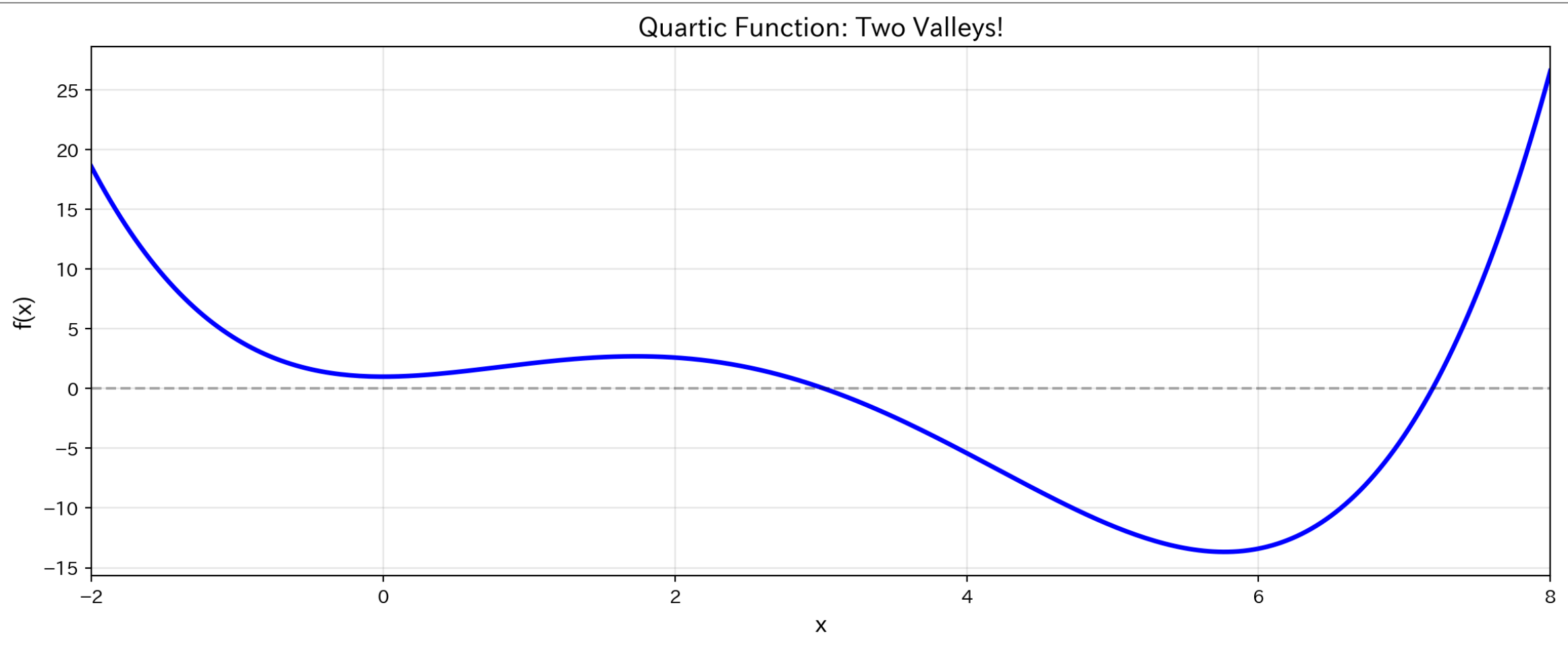
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 # 4次関数とその微分
6 def f_poly(x):
7     return 0.1*x**4 - x**3 + 2*x**2 + 1
8
9 def f_poly_prime(x):
10    return 0.4*x**3 - 3*x**2 + 4*x
11
12 def f_poly_double_prime(x):
13    return 1.2*x**2 - 6*x + 4
```

Python

7

まずグラフを描いてみよう

どんな形の関数なのか、**まず目で見**ることが大切です。



① グラフを観察しよう

谷がいくつあるか、数えてみてください。

- **左の谷** ($x \approx 1$ 付近)：浅い谷
- **右の谷** ($x \approx 6$ 付近)：深い谷

本編の2次関数は谷が1つだけでしたが、今回は**2つ**あります。

さて、Newton法を使ったら、どちらの谷に行くのでしょうか？

実験：初期値を変えてNewton法を実行してみよう

作戦：式は同じ、スタート地点だけ変える

Newton法 ([参考資料](#)) の更新式を思い出しましょう：

$$x_{\text{new}} = x - \frac{f'(x)}{f''(x)}$$

同じ式を使って、**スタート地点だけ変えて**2回実験します。

実験1： $x_0 = 0.5$ からスタート（左の谷の近く）

手計算で追いかけてみよう

反復1

現在の位置： $x = 0.5$

1次微分（傾き）を計算：

$$\begin{aligned} f'(0.5) &= 0.4 \times 0.5^3 - 3 \times 0.5^2 + 4 \times 0.5 \\ &= 0.05 - 0.75 + 2.0 = 1.3 \end{aligned}$$

2次微分（曲がり具合）を計算：

$$\begin{aligned} f''(0.5) &= 1.2 \times 0.5^2 - 6 \times 0.5 + 4 \\ &= 0.3 - 3.0 + 4.0 = 1.3 \end{aligned}$$

更新：

$$x_{\text{new}} = 0.5 - \frac{1.3}{1.3} = 0.5 - 1.0 = -0.5$$

反復2

現在の位置： $x = -0.5$ （谷底を飛び越した。でも大丈夫）

1次微分：

$$f'(-0.5) = 0.4 \times (-0.125) - 3 \times 0.25 + 4 \times (-0.5) = -0.05 - 0.75 - 2.0 = -2.8$$

2次微分：

$$f''(-0.5) = 1.2 \times 0.25 - 6 \times (-0.5) + 4 = 0.3 + 3.0 + 4.0 = 7.3$$

更新：

$$x_{\text{new}} = -0.5 - \frac{-2.8}{7.3} = -0.5 + 0.384 = -0.116$$

反復3

現在の位置： $x = -0.116$

$$f'(-0.116) \approx -0.51, \quad f''(-0.116) \approx 4.71$$

$$x_{\text{new}} = -0.116 + 0.108 = -0.009$$

反復4

$f'(-0.009)$ はほぼゼロ。収束しました！

❗ 実験1の結果

$x_0 = 0.5$ からスタート $\rightarrow x \approx 0$ （左の谷の底）に収束、 $f(0) = 1$

念のため $f''(0) = 4 > 0$ — ちゃんと谷です。

Pythonで確認

▶ コードを表示

実験1: $x_0 = 0.5$ からスタート

=====
反復 1: $x = 0.5000$, $f(x) = 1.3813$, $f'(x) = 1.3000$, $f''(x) = 1.3000$

更新: $x_{\text{new}} = 0.5000 - (1.3000)/(1.3000) = -0.5000$

反復 2: $x = -0.5000$, $f(x) = 1.6313$, $f'(x) = -2.8000$, $f''(x) = 7.3000$

更新: $x_{\text{new}} = -0.5000 - (-2.8000)/(7.3000) = -0.1164$
反復 3: $x = -0.1164$, $f(x) = 1.0287$, $f'(x) = -0.5071$, $f''(x) = 4.7149$
更新: $x_{\text{new}} = -0.1164 - (-0.5071)/(4.7149) = -0.0089$
反復 4: $x = -0.0089$, $f(x) = 1.0002$, $f'(x) = -0.0358$, $f''(x) = 4.0535$
更新: $x_{\text{new}} = -0.0089 - (-0.0358)/(4.0535) = -0.0001$
反復 5: $x = -0.0001$, $f(x) = 1.0000$, $f'(x) = -0.0002$, $f''(x) = 4.0004$

★ 収束! $x = -0.0001$, $f(x) = 1.0000$

実験2： $x_0 = 5.5$ からスタート（右の谷の近く）

手計算で追いかけてみよう

反復1

現在の位置: $x = 5.5$

1次微分:

$$\begin{aligned} f'(5.5) &= 0.4 \times 5.5^3 - 3 \times 5.5^2 + 4 \times 5.5 \\ &= 0.4 \times 166.375 - 3 \times 30.25 + 22.0 \\ &= 66.55 - 90.75 + 22.0 = -2.2 \end{aligned}$$

2次微分:

$$\begin{aligned} f''(5.5) &= 1.2 \times 5.5^2 - 6 \times 5.5 + 4 \\ &= 1.2 \times 30.25 - 33.0 + 4.0 \\ &= 36.3 - 33.0 + 4.0 = 7.3 \end{aligned}$$

更新:

$$x_{\text{new}} = 5.5 - \frac{-2.2}{7.3} = 5.5 + 0.301 = 5.801$$

反復2

現在の位置: $x = 5.801$

1次微分:

$$\begin{aligned} f'(5.801) &= 0.4 \times 5.801^3 - 3 \times 5.801^2 + 4 \times 5.801 \\ &= 78.11 - 100.95 + 23.20 = 0.36 \end{aligned}$$

2次微分:

$$\begin{aligned} f''(5.801) &= 1.2 \times 5.801^2 - 6 \times 5.801 + 4 \\ &= 40.37 - 34.81 + 4.0 = 9.56 \end{aligned}$$

更新:

$$x_{\text{new}} = 5.801 - \frac{0.36}{9.56} = 5.801 - 0.038 = 5.763$$

反復3

$f'(5.763)$ はほぼゼロ。収束しました！

❗ 実験2の結果

$x_0 = 5.5$ からスタート $\rightarrow x \approx 5.76$ に収束

$$f(5.76) \approx -3.04$$

Pythonで確認

▶ コードを表示

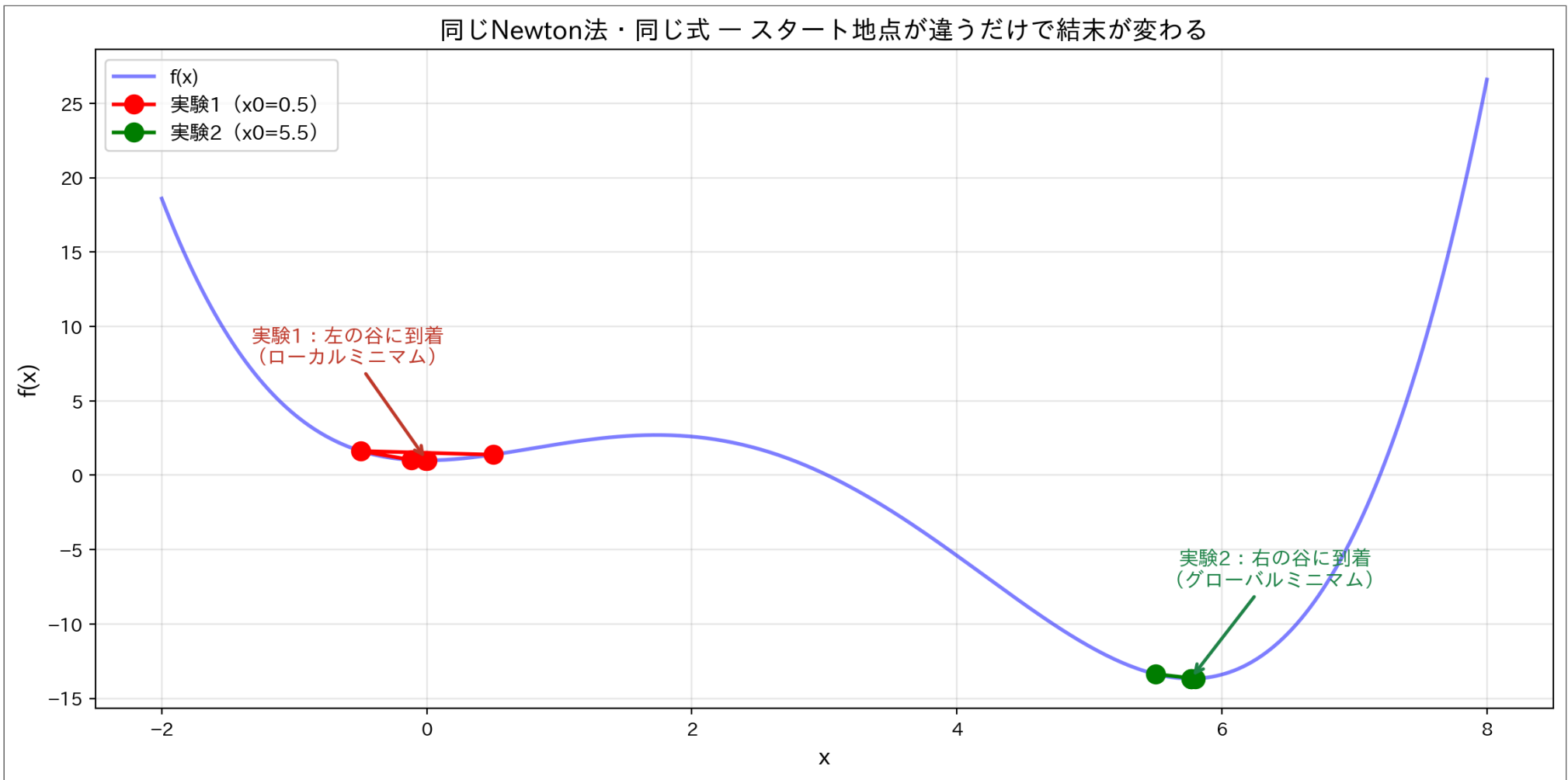
実験2: $x_0 = 5.5$ からスタート

```
=====
反復 1: x = 5.5000, f(x) = -13.3687, f'(x) = -2.2000, f''(x) = 7.3000
更新: x_new = 5.5000 - (-2.2000)/(7.3000) = 5.8014
反復 2: x = 5.8014, f(x) = -13.6666, f'(x) = 0.3379, f''(x) = 9.5789
更新: x_new = 5.8014 - (0.3379)/(9.5789) = 5.7661
```


反復 3: $x = 5.7661$, $f(x) = -13.6726$, $f'(x) = 0.0049$, $f''(x) = 9.3008$
更新: $x_{\text{new}} = 5.7661 - (0.0049)/(9.3008) = 5.7656$
反復 4: $x = 5.7656$, $f(x) = -13.6726$, $f'(x) = 0.0000$, $f''(x) = 9.2967$

★ 収束! $x = 5.7656$, $f(x) = -13.6726$

2つの実験を並べてみよう



⚠ たどり着いた谷が違う

まったく同じNewton法を使ったのに：

	スタート地点	到着地点	$f(x)$ の値	そこは何？
実験1	$x_0 = 0.5$	$x \approx -0.00$	1.00	谷（左）
実験2	$x_0 = 5.5$	$x \approx 5.77$	-13.67	谷（右・もっと深い）

- どちらも正真正銘の谷 ($f'' > 0$)。でも **深さがまったく違う**
- 実験1の答えは「その周辺では最適、全体では2番手」 — そして使った本人は、**もっと深い谷の存在に気づけない**

スタート地点（初期値）が違っただけで、たどり着く谷が変わってしまう。

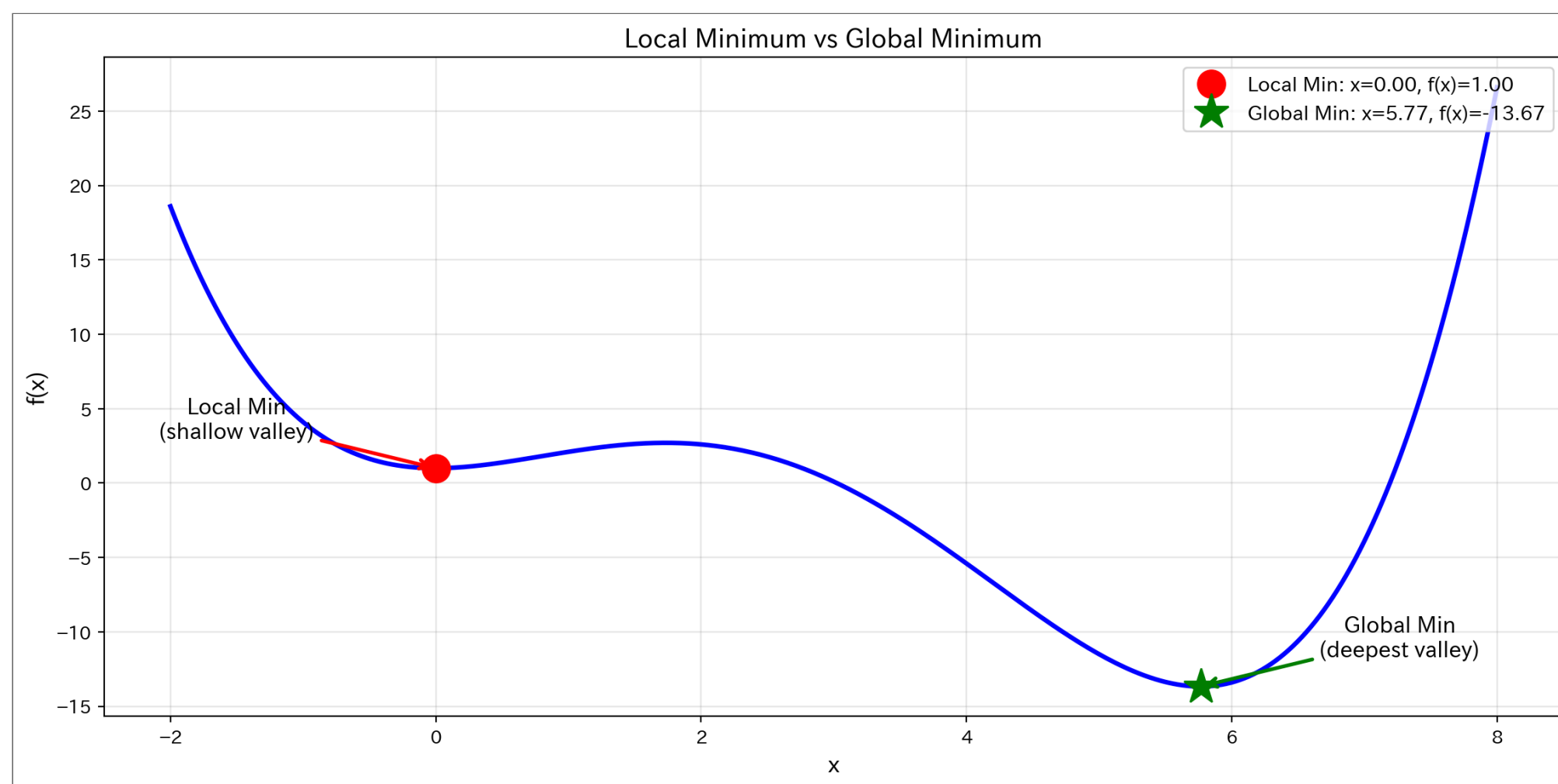
この現象に名前をつけよう

ローカルミニマムとグローバルミニマム

いま体験した現象には、ちゃんと名前がついています。

ローカルミニマム（局所的最小値, local minimum）：「その周辺では一番低い」けど、関数全体で見るともっと低い場所がある点。実験1で到達した**左の谷（ $x = 0$ ）**がこれです。

グローバルミニマム（大域的最小値, global minimum）：関数全体で**本当に一番低い**点。実験2で到達した**右の谷（ $x \approx 5.77$ ）**がこれです。



💡 たとえ話：山の中で一番低い場所を探す

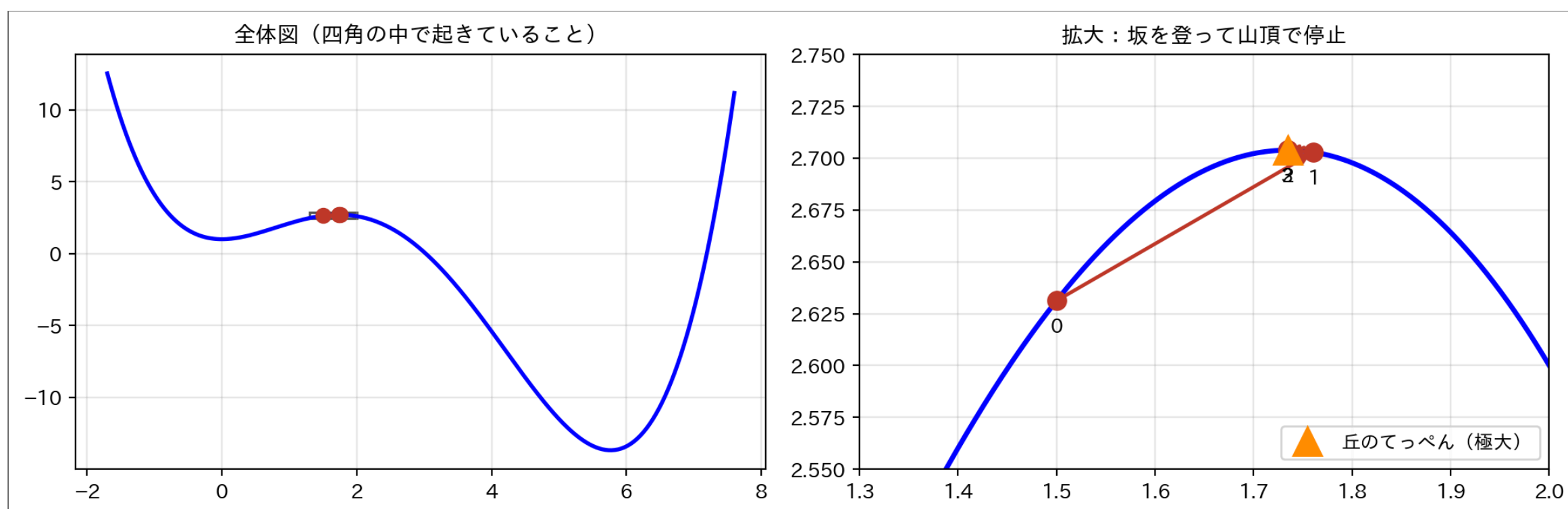
あなたは目隠しをして山の中にいます。足元の傾きだけを頼りに、一番低い場所を探します。

- 近くの谷に降りることはできる → **ローカルミニマム**
- でも、山の向こうにもっと深い谷があるかもしれない → **グローバルミニマム**
- 目隠しをしているので、**今いる谷が一番深いかわからない！**

Newton法も勾配降下法も、この「目隠し」と同じ状況です。

実験3：丘の中腹 $x_0 = 1.5$ から始めると...

- 同じ更新式を回すと： $1.5 \rightarrow 1.761 \rightarrow 1.735 \rightarrow$ 停止 ($f' \approx 0$)
- 左に下り坂があるのに、**坂を登って丘のてっぺんで止まった**



なぜ**登る**のか：

- 一步の向きは $-f'/f''$ 。ここで $f'(1.5) = +0.6$ (右上り坂)、 $f''(1.5) = -2.3$ (上に凸)
- 符号が打ち消し合って、一步は $+0.26$ — **傾きと同じ向き、つまり登る向き** になる
- Newton法が探しているのは「 $f' = 0$ の点」だけ。そこが山か谷かは**見ていない**
- 停止条件 ($|f'|$ がほぼゼロ) は山頂でも満たされる → **山で本当に止まる**

💡 f'' チェックは「止まった後の検品」

収束したら $f''(x)$ を確認：**正なら谷、負なら山** — 山なら結果を捨てる。山か谷かの区別は**できる**が、それは止まってからの話。走っている最中のNewton法には、登りを避けるブレーキがついていない。（実は後付けできる — 引力圏とカタパルトを見たあとで「修理」する）

16

なぜこうなるのか：初期値と「引力圏」

17

Newton法は「近くの傾きゼロ」に落ちる

Newton法は、現在地の近くにある「傾きゼロの点」を見つけるアルゴリズムです。遠くのことは知りませんし、そこが谷か山頂かも気にしません。

だから、だいたい：

- **左の谷の近くからスタート** → 左の谷（ローカルミニマム）に落ちる（実験1）
- **右の谷の近くからスタート** → 右の谷（グローバルミニマム）に落ちる（実験2）
- **丘の上からスタート** → 丘のてっぺんに登って止まる（実験3）

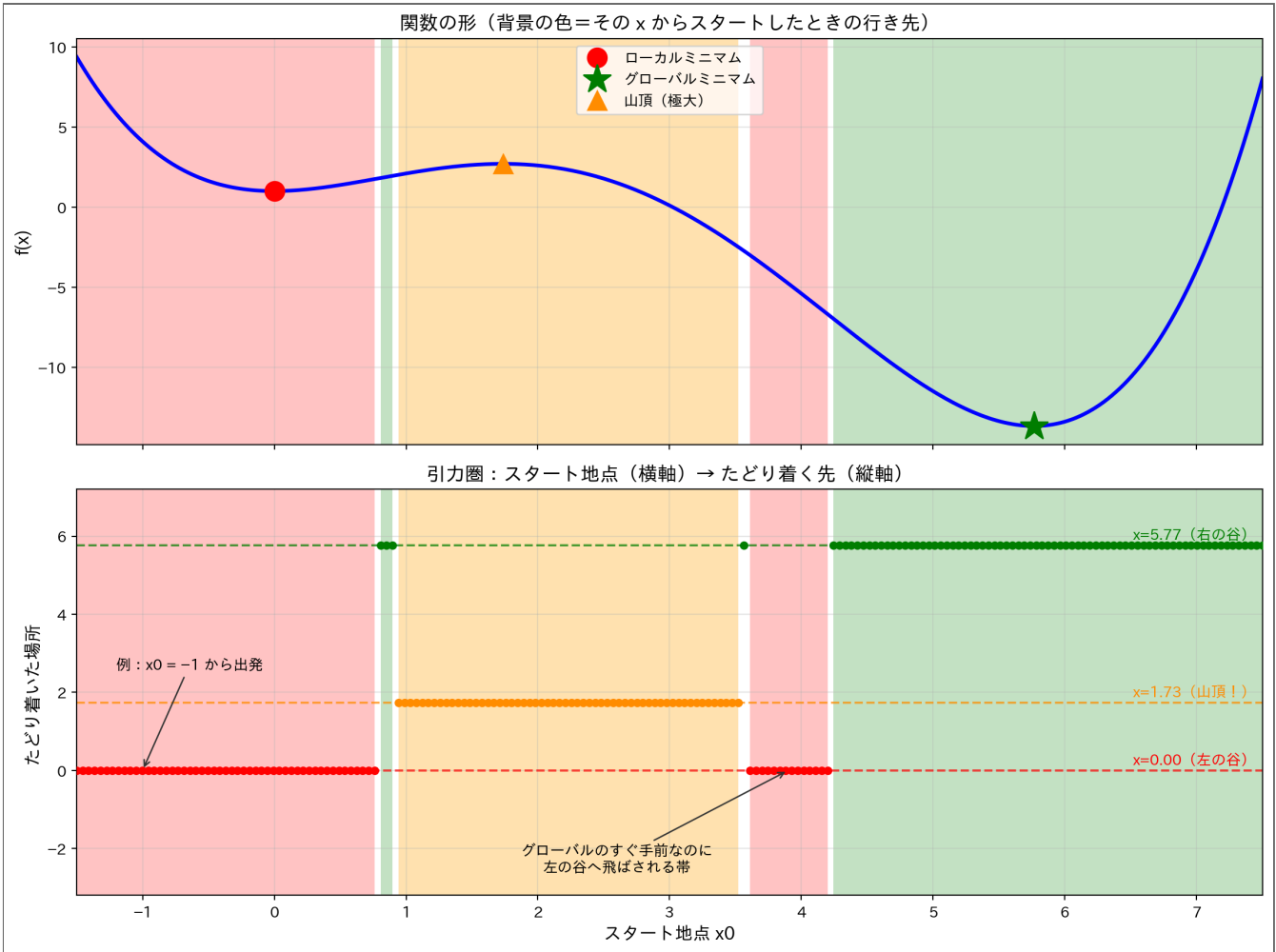
本当に「だいたい」で済むのか、初期値を200個並べて全部試してみましょう。

18

引力圏（Basin of Attraction）を可視化しよう

「どの初期値からスタートすると、どの谷に落ちるか」を全部調べてみましょう。

▶ コードを表示



図の見方（下のグラフ）：

- 横軸＝**スタート地点** x_0 、縦軸＝そこから Newton法が**最終的にたどり着いた場所**
- 縦に1本選んで読む：「 $x_0 = -1$ から出発 → 点の高さは 0 → 左の谷に着いた」
- 点は3つの高さのどれかに張り付く — 行き先は **左の谷（赤）・右の谷（緑）・山頂（橙）** の3択
- 背景の色の帯が**引力圏**：同じ色の帯から出発すると、同じ場所に吸い込まれる
- 上下のグラフは横軸が揃えてある — 上の山谷と下の帯を見比べられる

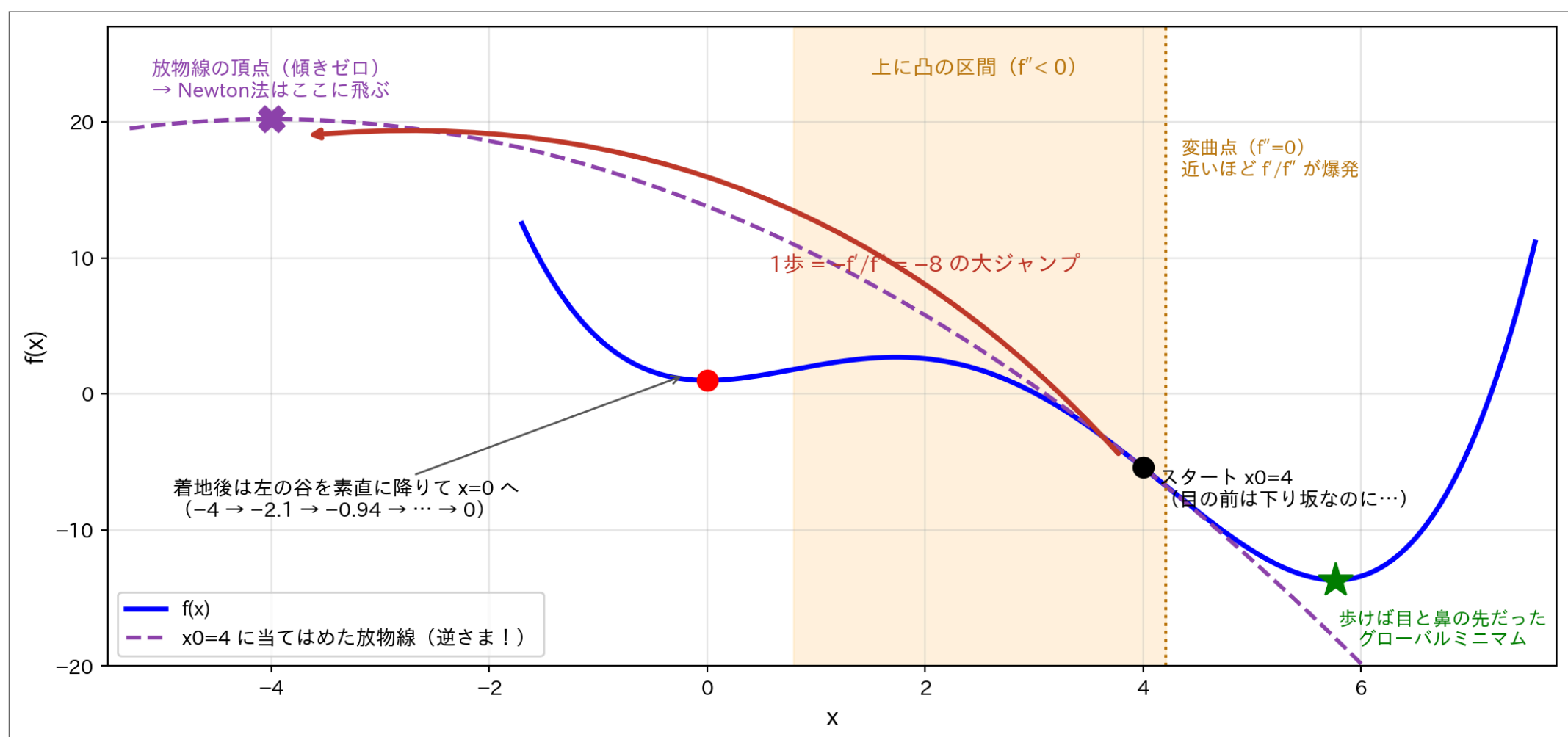
① 引力圏（Basin of Attraction）とは

各収束先には「引力圏」があります。その範囲からスタートすると、そこに吸い込まれる領域です。この実験では、素朴な予想より意地悪な結果になりました：

- 赤い帯 → ローカルミニマム、緑の帯 → グローバルミニマム、ここまでは予想どおり
- **橙の帯 → 山のとっぺん（極大）に到着して止まる**。Newton法が探しているのは「傾きゼロの場所」なので、谷と山頂を**区別できない**
- 帯はきれいな2分割ではない：**グローバルミニマムのすぐ手前（ $x_0 \approx 3.6 \sim 4.2$ ）から出発しても、左の谷へ飛ばされる**
- 初期値が少し変わるだけで行き先がガラッと変わる — 1回の実行を信用してはいけない理由がこれ

なぜ「すぐ手前の帯」が左へ飛ぶのか

$x_0 = 4.0$ から始めたときの**最初の1歩**を解剖してみます。



- Newton法の1歩は「いまの場所に**放物線を当てはめて、その頂点へジャンプ**」($x - f'/f''$ の意味)
- ところが $x_0=4$ は**上に凸の区間** (2つの変曲点 $x \approx 0.79 \sim 4.21$ の間) の右端 — 当てはめた放物線が**逆さま**になり、頂点は左の遠く ($x=-4$) にある
- しかも変曲点の近くでは $f'' \approx 0$ なので、割り算 f'/f'' が**爆発**して飛距離が伸びる
- 着地した $x=-4$ は左の谷の素直な (下に凸の) 区間 → あとはおとなしく $x=0$ に収束
- 細い緑の帯 ($x_0 \approx 0.85$) は同じ現象の**逆向き**：左の変曲点のすぐ内側から $+6.3$ ジャンプして右の谷へ

❗ 重要

大ジャンプできるのがNewton法の強みで、放物線近似が嘘をつく場所では同じ大ジャンプが暴発する。 勾配降下法なら「傾きの逆に少しずつ」なので、 $x_0=4$ からは素直に右の谷へ降りる。

— では、この壊れ方は直せないのか？ **直せます**。次で修理しましょう。

20

修理する：ワープの前に「検品」を入れる

21

故障の原因は「無条件ワープ」

ここまでに見た故障は2種類：

- **山頂で止まる**（実験3） — 上に凸の場所では、一步が**登る向き**になる
- **反対側へ吹っ飛ぶ**（カタパルト） — 変曲点の近くでは、一步が**大きすぎる**

原因はどちらも同じ。Newton法が、当てはめた放物線の頂点へ**検査なしでワープ**していることです。

💡 ヒント

修理方針：提案された一步を、**2つの検品**に通してから踏む。

22

検査1：その方向、本当に「下り」？

- Newton法の一步 $d = -f'/f''$ は**提案**として扱う
- 踏む前にチェック： $f' \cdot d \geq 0$ なら、提案は**傾きと同じ向き＝登り**なので**却下**
- 却下したら、確実に下る**最急降下方向** $d = -f'$ に差し替える
- 登り提案が出るのは、ちょうど $f'' < 0$ （上に凸）の場所 — 実験3とカタパルトの現場そのもの

実験3の現場検証： $x = 1.5$ では提案 $d = +0.26$ 。 $f' \cdot d = 0.6 \times 0.26 > 0 \rightarrow$ 登りなので却下 $\rightarrow d = -f' = -0.6$ （左の下りへ）

23

検査2：その一步、実測で下がってる？

- 向きが正しくても、**歩幅が大きすぎる**と谷を飛び越えて高い場所に着くことがある
- そこで、踏む前に着地点の**本物の f を実測**する：
 - 十分下がっていれば採用
 - 下がっていなければ**歩幅を半分**にしてやり直し（バックトラッキング）
- 放物線の「予言」ではなく、**実測**で確認してからしか進まない

24

修理版Newton法（検査1・2入り）

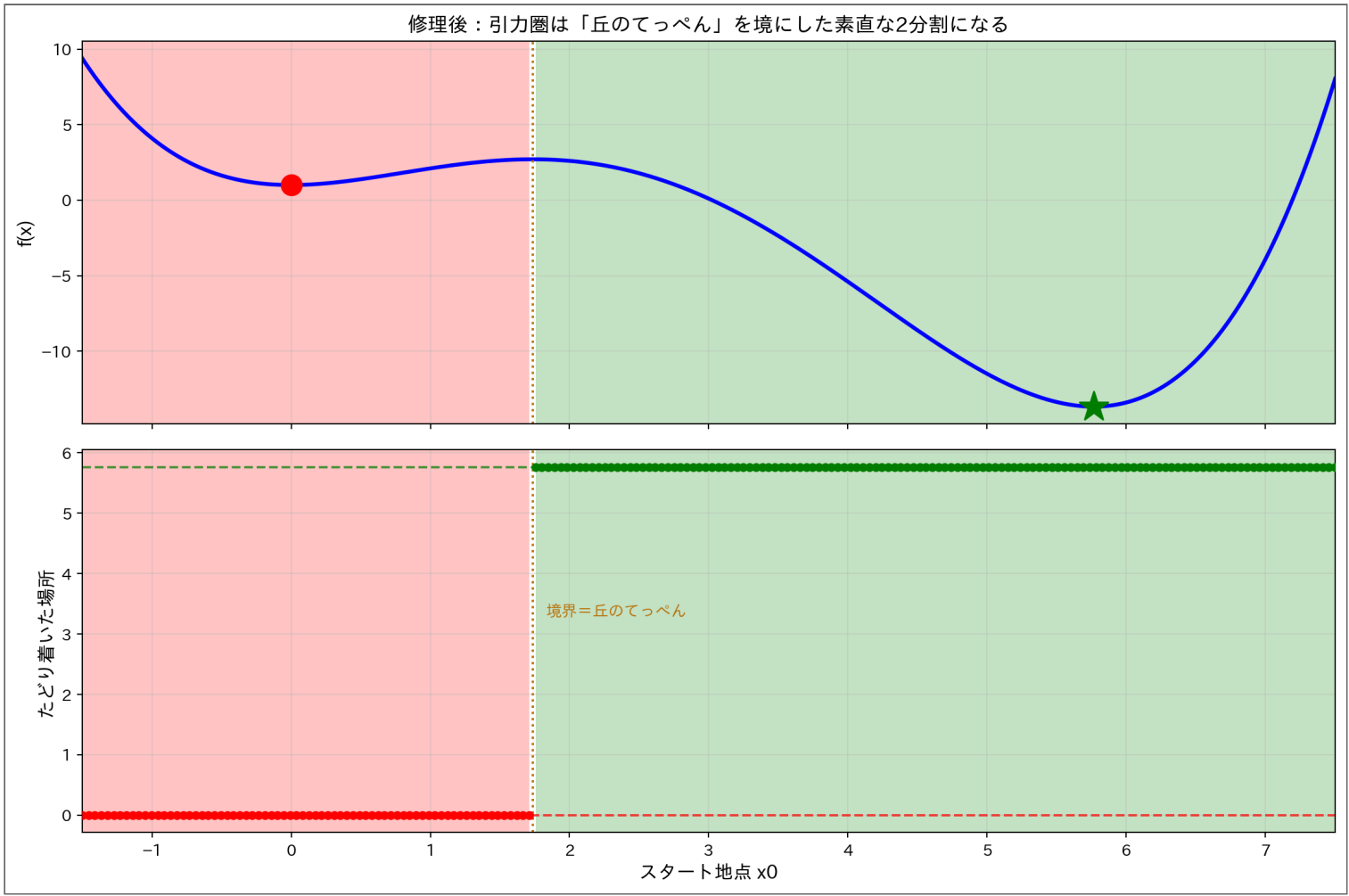
Python

```
1 def newton_safe(f, fp, fpp, x0, max_iter=100):
2     """Newton法+検査1（降下方向）+検査2（実測バックトラッキング）"""
3     x = x0
4     for _ in range(max_iter):
5         g = fp(x)
6         if abs(g) < 1e-8:                # 傾きほぼゼロ → 停止
7             break
8         h = fpp(x)
9         d = -g / h if h != 0 else -g      # Newton法の「提案」
10        if g * d >= 0:                    # 検査1：登りなら却下して
11            d = -g                        # 最急降下に差し替え
12        alpha = 1.0
13        while f(x + alpha * d) > f(x) - 1e-4 * alpha * abs(g * d):
14            alpha /= 2                    # 検査2：実測で下がるまで歩幅半分
15        x = x + alpha * d
16    return x, f(x)
17
18 # 故障していた2つの初期値で試運転
19 for x0 in [1.5, 4.0]:
20     x_fin, f_fin = newton_safe(f_poly, f_poly_prime, f_poly_double_prime, x0)
21     print(f"x0 = {x0} → x = {x_fin:.4f}, f(x) = {f_fin:.4f}")
```

x0 = 1.5 → x = -0.0000, f(x) = 1.0000
x0 = 4.0 → x = 5.7656, f(x) = -13.6726

- 山頂に登っていた $x_0 = 1.5$ は**左の谷**へ降りた
- カタパルトだった $x_0 = 4$ は、なんと**右の谷（グローバルミニマム）**へ — 暴発も消えた

修理後の引力圏



- 山頂の帯（橙）も、飛ばされる帯も消えた — てっぺんより左なら左の谷、右なら右の谷
- 検査1・2で直るのは、ここまで

ライブラリは最初から修理済み

- scipy などの実用ソルバーは、この種の検品を **毎歩** 組み込み済み：
 - **BFGS**：常に「下り向きの提案」しか出ないよう作った近似 f'' を使う
 - **Newton-CG**：負の曲率（上に凸）を検出した瞬間に計算を打ち切り、勾配方向へ切り替え
 - **信頼領域法**：放物線を「信じる半径」の中でだけ使う
- 実測： $x_0 = 1.5$ から **scipy.optimize.minimize** を呼ぶと、BFGS / Newton-CG / trust-exact のすべてが山頂に登らず $x = 0$ の谷に降りる
- 流儀は違ってても思想は同じ：「 f が下がらない一步は踏まない」

i ノート

授業で書いた素のNewton法が「欠陥品」なわけではない。教科書のNewton法はもともと「 $f' = 0$ を解く」方法で、**最小化に使うための安全装置**は別売り — ライブラリにはそれが標準装備されている、ということ。

27

対策：どうすればグローバルミニマムを見つけられるか

28

修理しても、直らないものがある

修理後の引力圏図をもう一度思い出すと：

- 「山に登る」「吹っ飛ぶ」は直った
- でも行き先は相変わらず **2つ** ある — **どっちの谷に落ちるかは初期値しだい**のまま
- これはNewton法の故障ではなく、**目隠しで坂を下ること自体の限界**（目の前の谷しか見えない）

解決策はシンプル：何回もやって、一番良い結果を選ぶ！

29

方法1：手で初期値を変えて比較する

まず、3つの初期値から手計算で比較してみましょう。

$x_0 = 0$ からスタート

$f'(0) = 0.4 \times 0 - 3 \times 0 + 4 \times 0 = 0$
 $f'(0) = 0$ なので、 $x = 0$ は既に極値です。 $f(0) = 1$ 。
 $f''(0) = 1.2 \times 0 - 6 \times 0 + 4 = 4 > 0 \rightarrow$ 最小値（谷）。

$x_0 = 2$ からスタート

$f'(2) = 0.4 \times 8 - 3 \times 4 + 4 \times 2 = 3.2 - 12 + 8 = -0.8$
 $f''(2) = 1.2 \times 4 - 6 \times 2 + 4 = 4.8 - 12 + 4 = -3.2$
Newton法の提案は $d = -\frac{-0.8}{-3.2} = -0.25$ （左へ）。でも傾きは右下り（ $f' < 0$ ）なので、
左へ進むのは登り — 検査1が却下して $d = -f' = +0.8$ （右へ）に差し替えます。
→ そのまま右の谷（グローバルミニマム）まで降りていきます。素のNewton法なら丘のてっぺん
行きでした。

$x_0 = 6$ からスタート

$f'(6) = 0.4 \times 216 - 3 \times 36 + 4 \times 6 = 86.4 - 108 + 24 = 2.4$
 $f''(6) = 1.2 \times 36 - 6 \times 6 + 4 = 43.2 - 36 + 4 = 11.2$
 $x_{\text{new}} = 6 - \frac{2.4}{11.2} = 6 - 0.214 = 5.786$
→ 右の谷（グローバルミニマム）の方向に進んでいます。

3つの結果を比較

```
1 # 3つの初期値から実行して比較
2 starting_points = [0, 2, 6]
3
4 print("初期値を変えて比較（修理版Newton法）")
5 print("=" * 60)
6 print(f"{'x0':>4} → {'到着地点':>8} → {'f(x)':>8} → どの谷？")
7 print("-" * 60)
8
9 results_comparison = []
10 for x0 in starting_points:
11     x_final, f_final = newton_safe(f_poly, f_poly_prime, f_poly_double_prime, x0)
12     results_comparison.append((x0, x_final, f_final))
13
14 best_f = min(r[2] for r in results_comparison)
15 for x0, xf, fx in results_comparison:
16     where = "左の谷" if xf < 3 else "右の谷"
17     marker = " ← 最良!" if fx == best_f else ""
18     print(f"{'x0':4.1f} → {'xf':8.4f} → {'fx':8.4f} → {where}{marker}")
```

初期値を変えて比較（修理版Newton法）

x0	→	到着地点	→	f(x)	→	どの谷？
0.0	→	0.0000	→	1.0000	→	左の谷

2.0 → 5.7656 → -13.6726 → 右の谷
6.0 → 5.7656 → -13.6726 → 右の谷 ← 最良！

❗ ポイント

修理版でも、3つ試すと**左右両方の谷**が出てきました。 $f(x)$ が**最小のもの**を採用する — これで一番良い答えを選べます。

手間は3倍になりますが、浅い谷の答えを信じてしまうよりずっと良いです。

方法2：たくさんの初期値を自動で試す（ランダムリスタート）

手で3つ試すのも良いですが、コンピュータなら**20個でも100個でも**試せます。

▶ コードを表示

ランダムリスタート：20個の初期値から試す

```
=====
試行 1: x0= 2.00 → x= 5.77, f(x)=-13.6726
試行 2: x0= 6.61 → x= 5.77, f(x)=-13.6726
試行 3: x0= 4.86 → x= 5.77, f(x)=-13.6726
試行 4: x0= 3.79 → x= 5.77, f(x)=-13.6726
試行 5: x0= 0.25 → x= -0.00, f(x)= 1.0000
... (残り15個)
```

★ 最良の結果: x0=2.00 → x=5.7656, f(x)=-13.6726

方法3：等間隔に初期値を並べる（グリッドサーチ）

ランダムではなく、**等間隔に並べる**方法もあります。

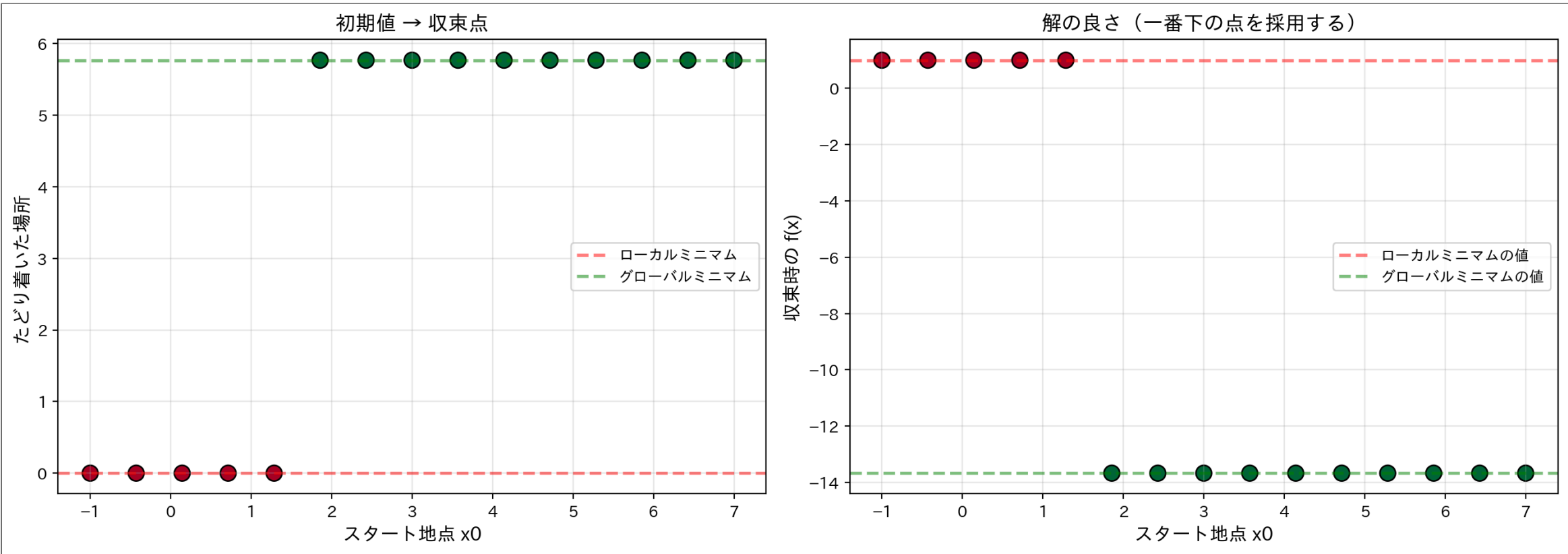
▶ コードを表示

グリッドサーチ：等間隔の15個の初期値

=====		
x0	→	到着地点 → f(x)

-1.00	→	-0.0000 → 1.0000
-0.43	→	-0.0000 → 1.0000
0.14	→	-0.0000 → 1.0000
0.71	→	-0.0000 → 1.0000
1.29	→	-0.0000 → 1.0000
1.86	→	5.7656 → -13.6726
2.43	→	5.7656 → -13.6726
3.00	→	5.7656 → -13.6726
3.57	→	5.7656 → -13.6726
4.14	→	5.7656 → -13.6726
4.71	→	5.7656 → -13.6726
5.29	→	5.7656 → -13.6726
5.86	→	5.7656 → -13.6726
6.43	→	5.7656 → -13.6726
7.00	→	5.7656 → -13.6726

★ 最良：x0=4.14 → x=5.7656, f(x)=-13.6726



💡 ランダムリスタート vs グリッドサーチ

方法	やり方	長所	短所
ランダムリスタート	初期値をランダムに選ぶ	簡単、偏りが少ない	運に左右される
グリッドサーチ	初期値を等間隔に並べる	漏れが少ない	数が多いと大変

どちらも「何回も試して一番良いのを選ぶ」という同じ考え方です。

最適化の4ステップ

最適化問題に取り組むときは、以下の手順を守りましょう。

① ステップ1：関数を可視化する

まずグラフを描いて、谷がいくつあるか確認する。谷が複数ありそうなら、ローカルミニマムに注意！

① ステップ2：複数の初期値を用意する

最低でも3～5個。余裕があれば10個以上。グリッドサーチかランダムリスタートで用意する。

① ステップ3：全部実行して、結果を比較する

すべての初期値からNewton法を実行し、 $f(x)$ が**一番小さい**結果を選ぶ。

① ステップ4：2次微分で確認する

収束した点で $f''(x) > 0$ なら谷（最小値）、 $f''(x) < 0$ なら山なので捨てる。 検査1・2入りの修理版ならまず山には止まらないが、確認は1行 — **保険として常に入れておく**（素のNewton法を使ったときは必須）。

実装テンプレート

上の4ステップをまとめたコードです。コピーして使えます。

[illegible]


```
60 print("すべての結果:")
61 print(f"{'x0':>4} → {'到着地点':>8} → {'f(x)':>8} → {'谷?':>4}")
62 print("-" * 50)
63 for r in all_results:
64     valley = "はい" if r['is_minimum'] else "いいえ"
65     marker = " ← 最良!" if r['x_final'] == best_x else ""
66     print(f"{r['x0']:4.1f} → {r['x_final']:8.4f} → {r['f_final']:8.4f} → {valley}{marker}")
67
68 print(f"\n★ 最良解: x = {best_x:.4f}, f(x) = {f_poly(best_x):.4f}")
```

すべての結果:

x0	→	到着地点	→	f(x)	→	谷?

-1.0	→	-0.0000	→	1.0000	→	はい
0.0	→	0.0000	→	1.0000	→	はい
2.0	→	5.7656	→	-13.6726	→	はい ← 最良!
4.0	→	5.7656	→	-13.6726	→	はい
6.0	→	5.7656	→	-13.6726	→	はい

★ 最良解: x = 5.7656, f(x) = -13.6726

まとめ

今回学んだこと

📌 ノート

- 1. 谷が複数ある関数では、Newton法の結果が初期値に依存する
 - 左の谷の近く → 左の谷、右の谷の近く → 右の谷
 - 丘の上からだと山頂に登って止まる ことすらある（素のNewton法は $f' = 0$ を探しているだけ）
- 2. ローカルミニマムとグローバルミニマム
 - ローカルミニマム = 周辺では一番低いが、全体では最低ではない
 - グローバルミニマム = 全体で本当に一番低い点
- 3. 修理できる故障と、できない故障がある
 - 検査1（降下方向か）＋検査2（実測で下がったか）で「山に登る・吹っ飛ぶ」は直る — ライブラリは標準装備
 - どの谷に落ちるか だけは、どんなに修理しても初期値しだい
- 4. 対策：複数の初期値から試して、一番良い結果を選ぶ
 - ランダムリスタートやグリッドサーチを使う
 - 2次微分の確認（ $f'' > 0$ なら谷）は保険として常に入れる